

Week 5 Assignment — Reflection

Intelligent Anomaly Detection and AI-Generated Root Cause Analysis

WHAT I BUILT

A small but complete pipeline: `generate_data.py` produces labelled synthetic metrics and correlated structured logs; `anomaly_detector.py` runs Isolation Forest (and DBSCAN as a comparison) with precision/recall/F1 against known ground truth; `alert_grouper.py` expands the 16 anomalous rows into 48 per-metric alerts and groups them by time-gap into a single incident; `rca_agent.py` runs an agentic RCA over that incident using two tools (`get_metrics`, `get_logs`) and a synthesis step, all instrumented with OpenTelemetry spans following the GenAI semantic conventions.

KEY DECISIONS

I chose Isolation Forest as the primary detector because `contamination` is a direct, interpretable prior on the anomaly rate. I chose rule-based (time-gap) grouping over LLM-based grouping because it is a deterministic problem that does not benefit from an LLM call — using an agent where a five-line rule suffices would be wasted cost and latency.

WHAT DIDN'T WORK AS EXPECTED — HONESTLY

- `uv` was not installed on the build machine; had to `pip install --user uv`, and the resulting executable was not on `PATH`, so every command ran as `python -m uv` instead of the documented bare `uv` command.
- `uv init` defaults to a packaged project layout (a build backend and an entry-point script), which is wrong for a set of standalone scripts. Had to delete the generated package and set `[tool.uv] package = false` before `uv sync` would stop trying to build a wheel.
- Notebook cells that shelled out to the agent scripts via `subprocess.run(["python", ...])` failed with `ModuleNotFoundError: No module named 'opentelemetry'`, even though the notebook kernel itself was running inside the project's venv. Cause: a bare `"python"` string resolves to whatever is first on `PATH`, not the venv interpreter. Fixed by using `sys.executable` instead — a good example of an assumption ("the subprocess inherits my environment") that is silently false inside a Jupyter kernel.
- DBSCAN badly underperformed Isolation Forest (F1 0.48 vs. 0.89) with a fixed `eps=1.0`, `min_samples=5`. This was a genuine surprise: the incident is a dense 16-point cluster, which is exactly what DBSCAN is supposed to find. The failure mode is that a dense *anomalous* cluster looks identical to a dense *normal* cluster from DBSCAN's point of view — density alone doesn't encode "far from normal," only "close to its neighbors." Isolation Forest's isolation-based scoring only cares about typicality relative to the whole dataset, which is the right inductive bias here.

WHAT I'D DO DIFFERENTLY WITH MORE TIME

Tune DBSCAN's `eps` / `min_samples` properly (e.g. via a k-distance plot) instead of a fixed guess, to see how close it can get to Isolation Forest; and wire the RCA agent's tools to a real Claude call behind a feature flag, so switching from simulated to live is a one-line config change rather than a rewrite. (No `ANTHROPIC_API_KEY` was configured in this environment, so all "LLM calls" in this submission are simulated but grounded in real detector/log data — see `README.md` for details.)